Universidad Nacional Autónoma de México

Computer engineering

COMPILERS

LEXER

STUDENTS:
321117744
321339009
321008215
321032308
320237012

Group: 5
Team: 4
Semester:
2026-II

México,CDMX. March 15th, 2026

Contents

1 Introduction **2**

2 Theorical Framework **2**

3 System Design **3**

3.1 main.py 4

3.2 AnalizadorLexico.py 4

3.3 Programs folder 4

4 Results **4**

5 Conclusions **5**

1 Introduction

- **Problem statement.** It will be carried out the creation of a Lexer, which will analyze a Context-Free Grammar pre defined.
- **Motivation.** Apply the concepts learned in class, related to the structure of the grammar, as well as how a Lexer works.
- **Objective.** Obtain a better understanding of the way a Lexer is structured, which is the first phase of a compiler.

2 Theorical Framework

Grammar explains the way in which the elements of the language relate to each other in order to create valid strings [1]. It also allows the analyse of the meanings that these combinations produce. Its types are:

- **Type zero ("Unrestricted Grammar").** This kind of grammar lacks structural restrictions. Due to its structure, it is not desirable for this project.
- **Type one ("Context-Sensitive Grammar").** A production can be applied depending on the surrounding context of a symbol. The length of the string produced is greater than or equal to the length of the string that is being replaced.
- **Type two ("Context-Free Grammar")**. It is a notation to express strings of a language as a function of simpler strings. It is defined by a quadruple:

  $$G = (V, \Sigma, S, P)$$

  Where

  V = Set of non-terminal symbols.

  Σ = Alphabet, terminal, or lexical symbols.

  S = Start symbol, where $S \in V$.

  P = Set of production rules. [2]
- **Type three ("Regular Grammar").** Their productions are restricted to right- linear or left-linear forms.

Type two and type three grammars are commonly used on compiler designs. In this project, we will use a predefined Context-Free Grammar.

A Lexer, also known as a Lexical Analyzer, is the first phase of analysis in a compiler. Its main function is to read the set of characters that make up a source program. The Lexer groups them into lexemes, that is, meaningful sequences; for

each lexeme, it produces, as an output, a token.

The most important points in lexical analysis are:

- **Tokenization.** The input characters are divided into tokens, that is, meaningful units, such as identifiers, keywords, constants and operators.
- **Regular expressions.** They are used to define token patterns. They allow the classification of programming language elements during tokenization.
- **Removal of whitespace**. Elements that are not relevant for the lexical analysis are removed, such as spaces, comments and tabulations.
- **Symbol table**. A symbol table is usually constructed during the analysis, which stores information about constants and identifiers found in the source code.
- **Detection of lexical errors.** The Lexer reports lexical errors, such as unknown words and symbols, as well as illegal characters.
- **Structured output.** At the end of the lexical analysis, a sequence of well defined tokens is obtained.

The completion of lexical analysis allows the beginning of the next phase: syntactic analysis. [3]

One of the libraries for implementing lexical analysis is RPLY. This is a pure Python parser generator that offers easy-to-use tools to indicate the rules needed for this type of analysis through a LexerGenerator instance, which represents a set of rules that match pieces of text that should either be turned into tokens or ignored by the Lexer. Rules are added using the method called add or ignored with 'ignore'.

3 System Design

The program relies on 3 main parts, the main file (main.py) that is the head of the project, used to manage and connect the different files and their respective outputs. The lexer (AnalizadorLexico.py) takes the source program that follow the grammar defined by it, reads the file and organizes every symbol it finds into tokens according to their individual definition. Finally the program folder holds the text file with extension .jp (Juan Pato files) that will then be the input for the respective analyzers.

3.1 main.py

Upon executing, the program asks the user to type the name of the file to be analyzed, if the form is left empty then the default program to access is "holamundo.jp", once the name is specified, the program tries to locate the specific file that should be found in the "programas" folder, if it isn't found then it prints on the screen that it failed to locate the file, otherwise it passes onto the analyzer home page. Here the user is able to either go ahead with the lexical analysis of the .jp file, or exit the compiler, if the Lexer is executed, it recovers the information coming from "AnalizadorLexico.py and outputs the list of tokens, their classification and the total amount found.

3.2 AnalizadorLexico.py

This is the file that carries the lexical analysis of the file, the first stage of the full compilation process. It employs the class lexer from the lexer generator library, this allows entries to be added to the lexer, by which we can define tokens with a name to identify each of them by, as well as the regular expression that the program expects to find in the .jp files and that it uses for cataloging each part of its contents. In doing so, we can define the grammar of the language for the compiler, and detect if a supposedly correct file follows the grammar properly or if it has lexical errors. It is able to classify all of the symbols and at the end of its execution, redirect the output to the main.py file to present the information to the user.

Some of the tokens that it is designed to accept are: the start of a program, the brackets that limit its span, the type of a variable, a variable name, a number, an operator (arithmetic, boolean, assignation), as well as what to ignore like white spaces and comments.

3.3 Programs folder

This folder that is on the same level as the files that compose the compiler, contains the text files of .jp extension which hold the programs to be analyzed by the Lexer, if it follows the grammar defined by it, then the compiler will output the list of tokens as well as the total amount, otherwise it will raise an error for it being lexically incorrect.

4 Results

To test the operation of the lexical analyzer, the following .jp program was used.

```
Start {
printf("This is an example");
int a = 10;
}
```

Figure 1: holamundo.jp

After running the lexical analyzer, the tokens in the example program were identified and their correct count was obtained. The following image shows the output of the lexical analyzer:

```
==================================================
                LEXICAL ANALYSIS STARTED
==================================================

Source Program:

----------------------------------------
Start {
printf("This is an example");
int a = 10;
}
----------------------------------------

Tokens Identified:

TOKEN TYPE           VALUE
----------------------------------------
START                Start
L_BRACKET            {
WRITING_OP           printf
L_PARENTHESYS        (
STRING               "This is an example"
R_PARENTHESYS        )
SEMI_COLON           ;
TYPE                 int
ID                   a
ASIG_OP              =
INTEGER              10
SEMI_COLON           ;
R_BRACKET            }
----------------------------------------
Total tokens identified: 13

Lexical analysis completed.
```

Figure 2: Output of the lexical analyzer - holamundo.jp

The list of identified tokens can be viewed in detail along with their respective classifications, as well as the total number of tokens in the given program; in this case, the total number of tokens was 13.

5 Conclusions

An analysis of the results obtained is presented, highlighting the importance of applying theoretical concepts to solve the problem. It is important to emphasize the following: Based on the obtained results, it can be observed that the theoretical concepts related to grammars and lexical analysis were correctly applied within the compilation process. By defining rules and patterns to identify tokens, the developed lexical analyzer was able to recognize and classify the different elements present in the input program. The results show that the tokenization process transforms a sequence of characters into an organized structure of tokens, which facilitates the subsequent analysis of the program. In this way, the importance of lexical analysis as the first stage in compiler design is demonstrated, since it allows the source code information to be structured for its processing in later stages of the compiler.

References

[1] Universidad Nacional Autónoma de México (UNAM). "La gramática (lo más básico)." LIEC. Accessed: 15-Mar-2026. [Online]. Available: https://liec.dgb.unam.mx/index.php/escribe/la-gramatica-lo-mas-basico.

[2] Universidad Nacional Autónoma de México (UNAM). "Gramáticas libres del contexto." Departamento de Ciencias de la Computación. Accessed: 15-Mar-2026. [Online]. Available: https://turing.iimas.unam.mx/~luis/cursos/ALF_old/sesiones/tema14.pdf.

[3] J. Ramirez. "Automatización y optimización de la generación de analizadores léxicos en la industria." ResearchGate. Accessed: 15-Mar-2026. [Online]. Available: https://www.researchgate.net/profile/Juan-Ramirez-Gramajo/publication/375160887_AUTOMATIZACION_Y_OPTIMIZACION_DE_LA_GENERACION_DE_ANALIZADORES_LEXICOS_EN_LA_INDUSTRIA/links/65425f930426ef6369f53749/AUTOMATIZACION-Y-OPTIMIZACION-DE-LA-GENERACION-DE-ANALIZADORES-LEXICOS-EN-LA-INDUSTRIA.pdf.